INTEGRAL DRAGON

MODULE 2 • SESSION 06

Plan Mode vs. Act Mode

Make the agent show its work before it edits the repo.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Mode separation is an engineering control, not a UI preference

- Planning and execution create different kinds of risk.
- The same agent that can help quickly can also widen scope quickly.
- Separating analysis from action gives the human a real approval gate — not a checkbox.

What Plan Mode is for

Read-only codebase analysis

Claude explores files, traces logic, and identifies boundaries without touching anything.

Multi-file change proposals

See the full proposed change surface before execution begins — with room to challenge it.

Clarifying question discovery

Missing requirements and hidden assumptions surface before a single line is written.

Safe exploration of sensitive areas

Auth, billing, compliance-sensitive paths — analyze the risk before committing to a direction.

Use Plan Mode when the task is high-risk or high-ambiguity

```
$ claude --permission-mode plan
```

- Multi-step implementation touching several files or systems
- Refactors that span multiple files or cross module boundaries
- Migrations, auth changes, billing logic, CI/CD, or compliance-sensitive flows
- Any work where the repo boundary is still unclear from the ticket alone

What a good plan should contain

Likely files and components touched

Explicit list — not "probably some auth files" but named paths and modules.

Invariants that must remain true

Schema stability, API contracts, test baseline, permissions model.

Migration or compatibility concerns

Any backward compatibility risk, rollout ordering, or flag dependency.

Testing strategy

Which tests will verify the change, what new tests are needed, and how to run them.

Open questions and assumptions

What the plan assumed without evidence — this is where reviewers should focus.

Review the plan like an engineer, not a spectator

- **What did the plan assume without evidence?** These are the highest-risk items.
- **What failure modes did it ignore?** Ask for explicit coverage of edge cases.
- **Which files should not be touched?** If they are not in the plan, confirm they are out of scope.
- **What test evidence would prove this change is safe?** Make it concrete before approving.

> **Approval is not a ceremony**
>
> A plan approval should be a defensible engineering judgment, not a rubber stamp. If you cannot explain why you approved it, the plan is not ready.

Claude Code has more than two permission modes — but one mental model governs them all

Mode	Behavior	When to use
`plan`	Read-only analysis; no repo writes	High-risk or high-ambiguity tasks
`default`	Prompts for approval on writes and commands	Normal development work
`acceptEdits`	Auto-approves file edits; prompts for commands	Trusted editing in known codebases
`auto`	Runs autonomously within allowed tool set	Scripted or pipeline-driven runs
`bypassPermissions`	No confirmations — full autonomy	Controlled automation only; never interactive

→ Core mental model: **analyze first, execute only after review.** The mode determines how that boundary is enforced.

The safest execution rhythm is small and explicit

01 Spec	02 Plan	03 Challenge the plan	04 Approve
05 Execute a bounded slice	06 Test	07 Narrate	→ Repeat

→ Each cycle should be reviewable. If a slice cannot be independently reviewed, it is too large.

LIVE DEMO

Propose → critique → refine → approve

Using Plan Mode on a real refactor before touching the repo

Prompt flow

```
$ claude --permission-mode plan

# Step 1 — explore
"Analyze the auth module and propose the safest refactor path."

# Step 2 — surface risk
"List touched files, hidden risks, and required tests."

# Step 3 — constrain
"Revise the plan to avoid schema changes and keep the diff small."
```

- → Did the plan name specific files? Did it acknowledge the risk it couldn't evaluate?
- → Could you approve this plan in a real code review? Would it hold up under scrutiny?

Common mistakes with Plan Mode

Approving a plan that never named the touched files

If you don't know what will change, you haven't reviewed the plan.

Letting the agent execute against ambiguity

An ambiguous request produces an ambiguous plan. Approve only when the acceptance criteria are clear.

Treating Plan Mode as ceremony

Running Plan Mode without reading the output doesn't add safety — it adds latency.

Using broad permissions to compensate for weak planning

`bypassPermissions` is not a substitute for understanding what the agent will do.

KEY TAKEAWAY

Planning is not delay.
Planning is where uncertainty leaves the repo.

Session 07 next: how persistent project rules in CLAUDE.md keep these constraints alive across every session.